metadexer - A High Level Overview

- **Project:** metadexer
- **Domain:** metadexer.com (reserved)
- **Author:** William Thompson (ShruggieTech LLC)
- **Date:** 2026-03-05
- **Supersedes:** `20260305-003-metadexer-project-direction.md`
- **Status:** APPROVED

Table of Contents

1. Audience
2. What Is `metadexer`?
3. The Problem
4. Who This Is For
5. Architecture
6. Real-World Scenarios
7. What Makes `metadexer` Different
8. Core Philosophy
9. Technical Foundation
10. Design Goals
11. Long-Term Extensibility
12. Development Roadmap
13. Naming and Branding
14. Summary
15. Document History
16. Appendix A: Uniform Conventions
17. Appendix B: Specification Status Lifecycle
18. Appendix C: Repository Organization
19. Appendix D: Archive Directory and Sprint Document Conventions
20. Appendix E: Implementation Guidance
21. Appendix F: Composition Rules

Audience

This document is written **AI-first, Human-second**.

It defines the vision, architectural intent, invariant boundaries, and uniform conventions for the metadexer project. Every section is written with sufficient detail for an AI implementation agent to produce correct design decisions within an isolated context window, without interactive clarification. Human developers and maintainers are the secondary audience - the document is equally valid as a traditional engineering reference and as a high-level concept overview.

What Is `metadexer`?

metadexer is a content-addressed asset management system for people who accumulate large, heterogeneous collections of digital data and need to store, deduplicate, catalog, and search across all of it - regardless of format, origin, or scale.

It answers one question: **"I have terabytes of files collected over years or decades. How do I organize them, find what I need, and never lose anything?"**

metadexer treats every file as an immutable object identified by its content, not its filename or path. It extracts rich metadata at ingestion time, stores the raw bytes in content-addressed vaults (local disk, S3-compatible buckets, or both), and maintains a searchable catalog of everything it has ever seen. Move a file, rename it, copy it to three different drives - metadexer knows it's the same object and tracks all of those observations over time.

This project exists because its lead developer needs it. The author maintains nearly 100TB of personal archives spanning two decades of collected data - videos, music, e-books, scraped websites, API feeds, OSINT research files, leaked datasets, and historical digital artifacts - distributed across cloud storage, local drives, and compressed archives. None of the existing tools were built to handle this. metadexer is.

The Problem

Managing a large, heterogeneous collection of digital assets across local filesystems, cloud storage, and archival media is fragile when done with ad-hoc tooling. The core failure modes are:

- **Identity is path-based.** Moving or renaming a file breaks every relationship that pointed to it. Two copies of the same file in different locations appear as unrelated objects.
- **Metadata is absent or siloed.** Most filesystems record a filename, a size, and three timestamps. Everything else - MIME type, embedded EXIF data, codec information, associated subtitle tracks, sidecar annotations - is either lost or locked in proprietary tool databases.
- **Deduplication is best-effort.** Manually running `fdupes` or `rdfind` after the fact is fragile, doesn't scale, and produces no durable record of what was deduplicated or why.
- **Integrity verification is manual.** There is no way to know if a file has been silently corrupted unless you hash everything yourself and maintain your own database of expected values.
- **Search is primitive.** Finding a specific video by a line of dialogue within it, or a music track by a lyric fragment, or an API response from a specific date and feed source, is effectively impossible with filesystem tools alone.

These problems compound. At 100 GB, they are annoying. At 10 TB or more, across multiple storage backends, with millions of files accumulated over a decade or two, they make the collection functionally unusable as anything other than a graveyard of data you vaguely remember saving once.

Who This Is For

metadexer is built for a specific kind of user. If you recognize yourself in any of the following, this tool is for you.

You are a data hoarder. You have terabytes of files on NAS drives, cloud storage, old hard drives in a drawer, and compressed archives you haven't opened in years. You know there are duplicates everywhere. You have no idea what's actually in half of it anymore. You have thought more than once about writing a script to "finally organize everything" and then realized the scope of the problem is larger than any script you could write in a weekend. You want something that will let you point it at a directory - any directory, any size - and have it indexed, deduplicated, and searchable without losing a single byte.

You are an OSINT researcher. You collect data on entities - governments, companies, individuals - from public sources, archived web pages, leaked databases, court filings, corporate registries, and social media scrapes. Your research files are scattered across folders organized by investigation, by date, by source, and by whatever system you were using at the time. You need to search across all of it at once. You need to know when you first observed a piece of data and where it came from. You need provenance that holds up under scrutiny. You need your archive to function as the institutional memory of a one-person intelligence operation.

You run a home lab and you are tired of your data being dumb. You have a Proxmox box or an Unraid server or a rack in a closet. You already run Ollama or vLLM or some other local inference stack. You have gigabytes - maybe terabytes - of personal documents, research notes, saved articles, technical references, and collected datasets that you wish your local LLM could actually search through and reason about. The missing piece has never been the model. It has been the retrieval layer - a structured, searchable, metadata-rich index of your own data that can feed context into a RAG pipeline without requiring you to first solve the problem of organizing everything by hand.

You collect media. You have thousands of videos, movies, TV shows, music albums, and audiobooks. You have subtitle files, lyrics files, NFO files, and cover art sitting alongside the media they describe. You want to search your video library by a line of dialogue. You want to find a song by a lyric fragment. You want all of that associated metadata to be indexed, linked to the parent media file, and queryable - not just sitting in the same folder and hoping for the best.

You run real-time data collection pipelines. You pull feeds from APIs - financial data, news wires, social media, market trackers, blockchain data - and write the responses to files. The files accumulate. The archives grow. You know there is signal buried in the noise, but you have no unified way to search across years of collected data, correlate events across sources, or build reports without manually extracting and grepping through archives.

You are a developer or sysadmin who has been burned by proprietary tools. You have used DAMs, CMSes, and asset management platforms that locked your metadata into opaque databases, charged per-seat, required a browser, and failed silently when you needed them most. You want something that runs from a terminal, stores everything in documented formats, and never makes a decision about your data that you did not explicitly authorize.

If none of the above describes you, metadexer is probably not what you need, and that is fine.

Architecture

metadexer consists of a standalone indexing tool and a unified application that handles storage, cataloging, and pipeline orchestration.

Component Map

```
shruggie-indexer (standalone tool, own repository)
    │
    │  produces IndexEntry v2 JSON
    │
    ▼
metadexer (single application, single repository)
    ├── vault module    - content-addressed byte storage
    ├── catalog module  - metadata registry, search, references
    └── sync module     - ingestion pipeline orchestration
```

The indexer is a standalone tool because it genuinely is one. It operates on local files, produces JSON output, has no dependency on metadexer, and is useful on its own - people can and will use it independently to index files, generate metadata, and pipe the output into their own scripts. It has its own repository and its own release schedule.

The vault, catalog, and sync modules share the IndexEntry contract, share configuration, share a data lifecycle, and are invoked together in normal operation. They live in a single repository as internal modules of the metadexer application. Each module maintains a clean responsibility boundary - the vault module does not know about catalog references, the catalog module does not store raw bytes - but they are not separate projects.

shruggie-indexer

Repository: shruggietech/shruggie-indexer

Release status: v0.1.2 (released 2026-03-05). Stable.

shruggie-indexer operates on local files and directories. It produces IndexEntry JSON records: structured metadata objects that serve as the authoritative description of a content object. It is the foundation on which all of metadexer is built.

What it does:

- Computes deterministic content identity (`id` , `storage_name`) via hash of file bytes.
- Extracts file metadata (MIME type, size, timestamps, format-specific fields via ExifTool).
- Produces well-formed IndexEntry v2 JSON output.
- Manages sidecar metadata files (discovery, parsing, MetaMergeDelete lifecycle).
- Provides session-level provenance (`session_id` , `indexed_at`) for downstream correlation.

What it does not do: storage management, reference tracking, orchestration, search, or anything involving a database or network.

Delivery surfaces: CLI (`click`), GUI (`customtkinter`), Python library (`index_path()` API).

Invariants:

- Given the same bytes, the same IndexEntry identity fields are always produced.
- IndexEntry output is the only artifact this tool emits. It does not side-effect storage.
- `session_id` links all entries from a single invocation; `indexed_at` records observation time distinct from file timestamps.

The authoritative technical specification is maintained separately in the shruggie-indexer repository (`shruggie-indexer-spec.md`). This direction document does not duplicate it.

metadexer

Repository: TBD (not yet created)

Release status: Pre-development.

metadexer is a single application with three internal modules. Each module has a clear responsibility boundary, but they share a repository, a configuration system, a test suite, and a release pipeline.

Vault Module

Content-addressed byte storage. Preserves raw file content.

What it does:

- Stores bytes under a deterministic key (`storage_name` from the IndexEntry).
- Retrieves bytes by `id` or `storage_name`.
- Checks existence without retrieval (head operation).
- Verifies stored bytes against a provided IndexEntry when explicitly requested.
- Prunes unreferenced objects when explicitly invoked.

Storage backends: local filesystem, S3-compatible object storage (AWS S3, MinIO, MEGA S4, etc.).

Invariants:

- The same `storage_name` always maps to identical bytes (write-once guarantee).
- The vault enforces identity; it does not compute it.
- Verification is always explicit - never triggered automatically during ingest.
- Pruning is always explicit - the vault never autonomously deletes content.

Catalog Module

Metadata registry and search engine. Records and indexes everything metadexer knows about every object.

What it does:

- Stores and retrieves full IndexEntry JSON records.
- Projects and indexes searchable fields (MIME type, size, timestamps, format-specific metadata, sidecar content, etc.).
- Tracks logical references to assets: collections, projects, users/tenants, snapshots.
- Provides search and query capability over indexed fields, including full-text search of string-stored content.
- Reconciles catalog contents against the vault (detects missing or orphaned blobs).
- Correlates IndexEntry snapshots across time using `id`, `session_id`, and `indexed_at` to build identity evolution history.
- Implements hybrid storage routing: stores small text-based content inline (as string data within the database) and routes large or binary content to the vault, based on configurable rulesets and explicit user direction.

Catalog database backends:

- **PostgreSQL** - the primary backend, built for power users running serious workloads. If you are managing millions of objects, running full-text search across years of ingested data, or operating metadexer as persistent infrastructure on a home server or dedicated machine, PostgreSQL is the intended database. This is the backend the lead developer uses. It is not an enterprise upsell - it is the default for anyone who takes their data seriously.
- **SQLite** - a lightweight alternative for quick evaluation, portable single-file deployments, and casual use. SQLite is suitable for getting started, running demonstrations, and managing smaller collections where the operational overhead of PostgreSQL is not justified. It is fully functional but not optimized for the concurrent access patterns or query complexity that arise at scale.

Invariants:

- `id` is globally unique per content object. Duplicate ingest is idempotent.
- Multiple references may point to a single asset; assets do not belong to references.
- Reference deletion removes the reference, not the underlying asset.
- Physical deletion from the vault requires a separate, explicit prune operation.

Catalog–Indexer contract: The IndexEntry is a point-in-time snapshot. Its fields describe a file's identity, metadata, and filesystem state at the moment of indexing. Over time, content hashes change when content is modified, timestamps shift through normal filesystem operations, metadata evolves as external tools and source files change, and relative paths change when files are moved or index roots differ between runs. This transient nature is correct by design. The indexer produces accurate snapshots; the catalog receives them, correlates them across time, and maintains a durable record of identity evolution.

Sync Module

Ingestion pipeline orchestrator. Connects the indexer, vault, and catalog into a reliable workflow.

What it does:

- Accepts directory or file targets as input.
- Invokes shruggie-indexer to produce IndexEntry records for each target.
- Generates Sync Plans (dry-run output of pending operations) before committing any changes.
- Checks the catalog and vault for already-present assets to avoid redundant work.
- Routes content to the appropriate storage destination (vault or inline catalog storage) based on rulesets.
- Uploads bytes to the vault.
- Commits IndexEntry records to the catalog only after upload is confirmed complete.
- Handles resumable uploads across interrupted runs.
- Supports dry-run mode with no side effects.

Operational modes:

- *Local mode:* Indexer → local vault → local catalog. No server required. Suitable for offline archival and single-user workflows.
- *Remote mode:* Client invokes indexer and computes identity locally. Client uploads bytes to remote vault. Client commits metadata to remote catalog. Server may verify integrity asynchronously by policy.

Invariants:

- Identity always originates from the client. Sync never delegates identity decisions to a server.
- Upload is confirmed complete before catalog commit.
- Catalog commit is idempotent. Resubmitting the same IndexEntry is safe.
- Sync is restartable at any point without risk of corruption or data loss.
- No asset is silently overwritten or deleted.

Real-World Scenarios

The following scenarios describe concrete workflows that metadexer is designed to support. These are not hypothetical product pitches. Each one reflects an actual use case that informed the system's design.

Scenario 1: Real-Time Feed Archival and Analysis

hotwire is a closed source real-time data collection system built by metadexer's lead developer. It pulls feeds from news providers, financial data APIs, social platforms, and market trackers. It writes each feed response to an individual file. Every few days, the accumulated files - typically 30,000 to 40,000 of them - are batch-exported into a compressed 7-zip archive. A single export contains around 107 distinct feed sources, with individual files ranging from a few kilobytes to over 20 GB. This pipeline has been running since approximately 2018.

Without metadexer, these archives are opaque. Finding a specific market event, or tracing how a story developed across news feeds over a particular week, requires manually extracting archives, navigating filesystem hierarchies, and grepping through individual files. There is no unified search, no cross-archive correlation, and no way to answer questions like "show me all globenewswire press releases from Q3 2024 that mention a specific company."

With metadexer:

1. **Index** - shruggie-indexer scans the extracted archive contents. Each file gets a deterministic content-addressed identity and rich metadata. Files that haven't changed since the last export are identified as duplicates by their content hash and skipped.
2. **Route** - metadexer's storage rules examine each IndexEntry. Small text-based API responses (JSON, CSV, XML) are routed to string storage within the catalog database for instant full-text search. Large binary files are routed to S3-compatible vault storage.
3. **Catalog** - every ingested object gets a catalog record with projected searchable fields. Feed source, timestamp, MIME type, size, and the full text content of string-stored objects become queryable. Session identity links all objects from a single export.
4. **Query** - "show me all globenewswire press releases from Q3 2024 mentioning Anthropic" becomes a catalog query against string-stored content, filtered by feed source prefix and timestamp range. The results include direct links to the stored content.

Over years of weekly exports, deduplication prevents the catalog from ballooning with redundant content that didn't change between collection cycles. Temporal tracking shows exactly when each piece of data was first observed.

Scenario 2: OSINT Research Operations

An independent researcher maintains an active investigation involving a network of shell companies, their registered agents, and their connections to politically exposed persons. The working dataset includes archived corporate registries, scraped public filings, cached court documents, exported social media profiles, local copies

of ICIJ leak data, and hundreds of annotated PDF reports. The files are organized by investigation name, but cross-investigation searches are impossible because the same entity appears in multiple cases under different folder hierarchies.

With metadexer, the entire research archive is ingested once. Content-addressed identity means that a PDF of the same court filing downloaded for three separate investigations exists as one object with three reference paths in the catalog. Sidecar annotation files - the researcher's own notes, tagged with entity names and dates - are indexed alongside the primary documents they describe. A single catalog query across all investigations returns every document, filing, and annotation that mentions a target entity name, regardless of which investigation folder it was originally filed under.

Temporal tracking records when each document was first ingested and from which source directory. This provenance chain answers the question "when did I first obtain this document and where did it come from?" - a question that matters when your research needs to be defensible.

The catalog effectively becomes the researcher's institutional memory. Every document ever collected, every annotation ever written, every observation ever recorded - searchable, cross-referenced, and traceable. One person, operating with the organizational infrastructure of an intelligence directorate.

Scenario 3: Local AI Knowledge Base (RAG Pipeline)

A developer runs Ollama on a home server with a 70B parameter model. They have accumulated several terabytes of personal data they want the model to be able to reference: technical documentation, saved research papers, project notes, years of bookmarked articles saved as PDFs, exported Slack and Discord logs, and collected code repositories. The perennial problem with local RAG setups is not the model and it is not the vector store - it is the retrieval layer. Getting your data organized, deduplicated, and searchable in a structured way is the prerequisite that most RAG tutorials skip over with "and then you chunk your documents."

metadexer is that prerequisite. After ingestion, the catalog contains a structured, searchable index of every document with full metadata: MIME type, timestamps, extracted text content (for string-stored documents), sidecar annotations, and format-specific fields. A RAG pipeline built on top of metadexer works as follows:

1. User submits a query to the application layer.
2. The application translates the query into a metadexer catalog search - full-text search over string-stored content, filtered by MIME type, date range, source collection, or any other indexed field.
3. metadexer returns matching IndexEntry records with content (inline for string-stored objects) or vault references (for binary objects that can be fetched and chunked on demand).
4. The application assembles the retrieved content into a context window and passes it to Ollama alongside the original query.
5. The model generates a response grounded in the user's actual data.

This is not a theoretical workflow. metadexer's hybrid storage model means that text-heavy documents (the most useful for RAG) are already stored inline in the catalog database, queryable by content without a separate vector store. For users who want semantic similarity search, an embedding layer can be added as a catalog

extension that reads from the same indexed content - metadexer handles the storage, identity, and structured retrieval; the vector layer handles the similarity matching.

The result: your local LLM can search through and reason about your entire personal archive. Not a curated subset you manually prepared. All of it.

Scenario 4: Media Library with Deep Search

A collector has approximately 8,000 video files (movies, TV series, documentaries, recorded lectures), 40,000 music tracks, and 12,000 audiobooks. Most videos have subtitle files (.srt, .vtt) in the same directory. Many music tracks have .lrc lyric files or .txt liner notes. Audiobooks have chapter metadata in sidecar files.

Without metadexer, searching this collection means relying on filenames and folder structure. Finding "that documentary where the narrator talks about the 1971 Nixon shock" requires remembering which folder it is in or scrubbing through files manually.

With metadexer, subtitle files are discovered during indexing as sidecar metadata and their text content is parsed into the parent video's MetadataEntry array. The catalog indexes this content. A search for "Nixon shock 1971" across the video collection queries the sidecar text data and returns the specific video file - along with the timestamp context from the subtitle file that matched.

The same principle applies to music (search by lyric fragment returns the track), audiobooks (search by chapter title or description), and any other media format where associated text files exist alongside the primary content. The sidecar metadata model turns every associated file into a searchable facet of the parent object, without requiring any special handling per format.

Scenario 5: Legacy Archive Migration and Deduplication

A sysadmin has inherited responsibility for a departmental file server containing 14 TB of data accumulated over 15 years across five successive storage migrations. The same files exist in multiple locations under different names. Folder structures from the original server, the first NAS, the second NAS, and the current NAS all coexist in a nested hierarchy that no one fully understands. There are at least three copies of everything that was considered important in 2012 and zero copies of some things that turned out to be important in 2025. There is no inventory. There is no deduplication. There is no way to answer "do we still have the original version of X?"

metadexer's content-addressed identity model resolves this directly. Indexing the entire file server produces an IndexEntry for every file. Files with identical content - regardless of name, path, or timestamp - resolve to the same `id`. The catalog records every path at which each unique object was observed, providing a complete map of duplication. A reconciliation report shows exactly how many unique objects exist, how many duplicate copies exist, and where they are located. Migration to clean, deduplicated vault storage can then proceed with full confidence that nothing has been lost, because the catalog maintains the complete observation history.

What Makes `metadexer` Different

Existing solutions fall into several categories, none of which solve this problem:

Traditional Digital Asset Management systems (ResourceSpace, Nuxeo, Pimcore, Razuna) are built for marketing teams managing brand assets. They optimize for approval workflows, brand consistency, and asset distribution to web channels. They use database-assigned or path-based identity. They have no concept of content-addressed deduplication, no hybrid storage tiering, no sidecar metadata model, and no API feed ingestion story. They are designed for teams managing thousands of curated assets, not for individuals managing millions of heterogeneous files.

Self-hosted archiving tools (ArchiveBox, Hydrus Network) are the closest in spirit. ArchiveBox is excellent for archiving web pages but is URL-centric, uses timestamp-based identity, has no real deduplication, and its metadata model is flat. Hydrus Network is a powerful media tagging application but is narrowly focused on image collections, has no S3 backend, no API feed ingestion, and no structured metadata schema.

OSINT collection and analysis tools (SpiderFoot, Maltego, Intelligence X) focus on finding and correlating information. None of them solve the durable storage, identity, and retrieval problem for data already collected.

General-purpose object storage (MinIO, raw S3) handles byte storage but provides no metadata indexing, no search, no deduplication logic, and no ingestion pipeline.

The specific capabilities that metadexer provides and none of these tools do:

1. **Content-addressed identity as the foundation.** Every object is identified by a hash of its bytes. Deduplication is deterministic and automatic. Moving or renaming files does not break anything. Two identical files ingested from different sources resolve to the same identity.

2. **A sidecar metadata model that makes associated content searchable.** Subtitle files associated with a video become part of that video's catalog record. Lyrics associated with a music track become searchable. Annotations and supplementary files associated with any object are discovered, parsed, and indexed alongside the primary content.

3. **Hybrid storage with rule-based routing.** A 500-byte JSON response from an API feed belongs in a database column for instant text search. A 4 GB video belongs in an S3 bucket. Both are tracked by the same catalog with the same identity model. Storage routing is determined by configurable rulesets and by explicit user direction at ingestion time.

4. **Temporal observation tracking.** Every IndexEntry is a point-in-time snapshot. The catalog correlates snapshots over time using content identity, session identity, and observation time. metadexer builds a history of when objects were observed, where they were located, and how their metadata evolved.

5. **Offline-first, CLI-first, composable.** metadexer runs entirely locally without network access. It is operated from a terminal. Every operation can be scripted, automated, and piped. It does not require a web browser, a running server, or an internet connection for core functionality.

Core Philosophy

These principles govern all design decisions. They are not aspirational - they are invariants.

Identity Is Deterministic and Client-Originated

All content identity - hashes, IDs, storage names - is computed client-side from the content itself. The indexer defines identity. metadexer enforces and records it. Servers do not define identity. Servers may verify identity periodically or probabilistically. The IndexEntry is the authoritative metadata record for a content object.

This system is **content-addressed**, not path-addressed.

Bytes and Metadata Are Separate but Linked

Each concern has exactly one home:

Concern	Owner
Identity generation and metadata extraction	shruggie-indexer
Raw byte storage	metadexer vault module
Structured metadata, references, and search	metadexer catalog module
Pipeline orchestration	metadexer sync module

No module is permitted to silently absorb a neighbor's responsibility.

Servers Are Registries and Policy Engines, Not Sources of Truth

Servers enforce authentication, record references, apply storage policies, and optionally verify integrity. Servers do not define or recompute content identity, mutate content or rewrite metadata, or make autonomous decisions about asset disposition.

Explicit Over Implicit, Always

All destructive operations (delete, prune, overwrite) require explicit opt-in flags. No destructive action is ever a default. Verification is explicit - never triggered automatically during ingest. Pruning is explicit - metadexer never autonomously deletes content. Failures surface explicitly - the system does not paper over errors.

Technical Foundation

The IndexEntry Contract

The IndexEntry v2 JSON schema is the stable foundation on which everything is built. It is defined and owned by shruggie-indexer. metadexer consumes it as a fixed contract and MUST NOT redefine its fields or semantics.

Canonical Schema Location

The authoritative machine-readable schema is hosted at:

```
https://schemas.shruggie.tech/data/shruggie-indexer-v2.schema.json
```

This document uses JSON Schema Draft-07. A local copy is committed to the indexer repository at `docs/schema/shruggie-indexer-v2.schema.json` and MUST be kept in sync with the canonical hosted version.

Schema Design Principles

P1 - Logical grouping. Related fields are consolidated into typed sub-objects rather than scattered across the top level.

P2 - Single discriminator for item type. A `type` enum (`"file"` or `"directory"`) combined with `schema_version` allows consumers to route parsing unambiguously.

P3 - Provenance tracking for metadata entries. MetadataEntry includes `origin` , `file_system` , `size` , `timestamps` , and an `attributes` sub-object - enough to reconstruct the original sidecar file.

P4 - Elimination of redundancy. No structurally redundant, platform-specific, or algorithmically redundant fields.

P5 - Explicit algorithm selection. `id_algorithm` records which hash algorithm produced `id` , making identity derivation fully self-describing.

Top-Level IndexEntry Fields

Property	Type	Required	Description
`schema_version`	`integer` (const `2`)	Yes	Always `2` . Placed first by convention.
`id`	`string`	Yes	Primary identifier. Prefix `y` = file, `x` = directory. Pattern: `^[xy][0-9A-F]+$` .
`id_algorithm`	`string` (enum)	Yes	`"md5"` or `"sha256"` .
`type`	`string` (enum)	Yes	`"file"` or `"directory"` .
`name`	`NameObject`	Yes	Item name with associated hash digests.
`extension`	`string` or `null`	Yes	Lowercase, no leading dot. `null` for directories.
`mime_type`	`string` or `null`	No	MIME type via `mimetypes.guess_type()` , overridden by ExifTool.
`size`	`SizeObject`	Yes	Human-readable and byte-count size.

Property	Type	Required	Description
`hashes`	`HashSet` or `null`	Yes	Content hashes (file) or `null` (directory).
`file_system`	`object`	Yes	Relative path and parent directory identity.
`timestamps`	`TimestampsObject`	Yes	Created, modified, and accessed timestamps (each as TimestampPair).
`attributes`	`object`	Yes	Symlink status and deterministic `storage_name`.
`items`	`array[IndexEntry]` or `null`	No	Child entries (directory) or `null` (file).
`metadata`	`array[MetadataEntry]` or `null`	No	Sidecar and generated metadata.
`session_id`	`string` or `null`	No	UUID4 linking to the invocation that produced this entry.
`indexed_at`	`TimestampPair` or `null`	No	When the entry was constructed (distinct from file timestamps).

`additionalProperties` is `false` at the root level - no extra keys are permitted.

Reusable Type Definitions

Type	Purpose
`NameObject`	Pairs a `text` string with its `hashes` (HashSet).
`HashSet`	Contains `md5`, `sha256`, and optionally `sha512` hex digests.
`SizeObject`	Pairs a human-readable `display` string with a `bytes` integer.
`TimestampPair`	Pairs an `iso` (ISO 8601) string with a `unix` (milliseconds) integer.
`TimestampsObject`	Groups `created`, `modified`, and `accessed` as TimestampPair objects.
`ParentObject`	Groups the parent directory's `id` and `name` (NameObject).
`MetadataEntry`	A sidecar or generated metadata record with `id`, `origin`, `name`, `hashes`, `attributes`, and `data`.

Schema Evolution Rules

- **Additive fields are non-breaking.** New optional fields MAY be added to v2 without incrementing `schema_version`. Consumers MUST tolerate unknown optional fields.
- **Structural changes require a version bump.** Renaming, retyping, removing a required field, or altering semantic meaning constitutes a breaking change and MUST increment `schema_version`.
- **Deprecation before removal.** A field marked deprecated in version N is emitted but ignored, then removed in version N+1.
- **Consumers dispatch on `schema_version`.** The integer value `2` is checked before parsing. Documents with unrecognized versions SHOULD be rejected.

Reference and Deletion Model

Assets are **immutable**. References are **mutable**.

Deletion is always two-phase:

1. Remove references in the catalog.
2. Explicitly invoke vault prune for unreferenced objects.

This model prevents accidental data loss. No content is ever removed in a single implicit operation.

Verification Philosophy

Integrity is enforced through **explicit verification**, not blind trust during ingest.

Verification modes available to the vault module:

- **Strict** - re-hash the entire stored object and compare against the IndexEntry.
- **Sampled** - probabilistic audits across a fraction of stored objects.
- **Tiered** - frequency and depth determined by policy or asset risk classification.

Verification is invoked by policy or on-demand. It is not part of the ingest critical path. This preserves ingestion performance while maintaining the ability to audit integrity continuously.

Failure Model

metadexer must tolerate and recover from interrupted or partial uploads, duplicate catalog commits, partial batch failures, network failures at any stage, and temporary unavailability of either the catalog database or vault storage.

The system always favors:

- **Consistency over convenience** - incomplete operations leave the system in a known, recoverable state.
- **Explicit reconciliation over implicit repair** - the system surfaces discrepancies and requires deliberate action to resolve them.

Design Goals

The following properties must be preserved across all future implementation decisions:

- **Determinism** - the same input always produces the same output.
- **Idempotence** - repeating an operation produces no additional side effects.
- **Composability** - the indexer works independently; metadexer modules work as a coordinated unit.
- **Auditability** - all operations are traceable; no silent mutations or deletions.
- **Minimal hidden state** - system state is observable and recoverable from durable records.
- **Clear failure modes** - failures surface explicitly; the system does not paper over errors.
- **Offline-first capability** - local operation is fully functional without network access.

Long-Term Extensibility

The following capabilities should be achievable through future extension without requiring changes to the identity model or core contracts:

- Multiple vault backends operating simultaneously.
- Multiple catalog instances (e.g., per-tenant, per-project).
- Vault-to-vault migration with integrity verification.
- Manifest export and import for portable asset sets.
- Snapshot materialization from catalog state.
- Immutable archival tiers with tiered access policies.
- A web-based UI layer for search and browsing (API-driven, thin over CLI logic).
- Integration with hotwire for automated real-time feed ingestion.
- Embedding and vector search extensions for semantic retrieval.

None of these capabilities require redefining content identity. The IndexEntry contract is the stable foundation on which all future extension is built.

Development Roadmap

The following sequence reflects the actual dependency order and the minimum viable path to a working system.

Phase 1: Foundation (Complete)

shruggie-indexer v0.1.2 is released and stable. It defines identity, extracts metadata, manages sidecars, and produces well-formed IndexEntry v2 JSON. This is done.

Phase 2: Storage and Catalog (Next)

Build the metadexer application with vault and catalog modules. Target: a working local-mode system where `metadexer ingest <directory>` indexes files, stores bytes in a local vault, and commits metadata to a catalog with basic query support.

- Vault module: put, get, head, verify. Local filesystem backend first. S3-compatible backend second.
- Catalog module: IndexEntry ingest, field projection, basic search (MIME type, size, timestamps, name, extension). PostgreSQL backend as primary target. SQLite backend for portable/evaluation use. Hybrid string storage for small text-based content.
- Skip for v0.1.0: reference tracking (collections/projects/users), temporal correlation, reconciliation, pruning.

Phase 3: Pipeline (Following)

Build the sync module. Target: a reliable, resumable ingestion pipeline with dry-run support.

- Sync Plan generation (dry-run preview of pending operations).
- Deduplication checks against existing catalog/vault state.
- Storage routing based on configurable rulesets.
- Resumable operation across interrupted runs.
- Idempotent catalog commits.

Phase 4: Search and Scale

Expand catalog capabilities to support the full intended feature set.

- Full-text search over string-stored content.
- Reference tracking (collections, projects, tenants).
- Temporal correlation of IndexEntry snapshots.
- Vault reconciliation (detect orphaned or missing blobs).
- Vault pruning for unreferenced objects.

Phase 5: Integration and Polish

- hotwire integration (automated feed ingestion pipeline).
- Web UI for search and browsing (thin layer over API).
- MEGA S4 as a vault backend.
- Documentation site, public release, metadexer.com.

Naming and Branding

metadexer is the product name. It is what users install, what the CLI is called, what the documentation refers to, and what goes on metadexer.com.

shruggie-indexer retains its current name and branding. It is an independent tool published under the ShruggieTech name that predates the metadexer product identity. It is a dependency of metadexer, not a sub-component of it. It has its own repository, its own release schedule, and its own documentation.

ShruggieTech is the organizational identity (ShruggieTech LLC). It is the publisher of both shruggie-indexer and metadexer. It appears in license headers, copyright notices, and GitHub organization naming. It is not a product name.

The `shruggie-tech` namespace for application data directories is shared between shruggie-indexer and metadexer:

```
<platform_base>/shruggie-tech/
├── shared.toml                  # (future: cross-tool configuration)
├── shruggie-indexer/
│   ├── config.toml
│   ├── gui-session.json
│   └── logs/
└── metadexer/
    ├── config.toml
    └── logs/
```

Summary

> The **indexer** defines truth.
> **metadexer** preserves, records, and propagates it.

No component is permitted to redefine it.

Document History

Date	Change
2026-03-05	Initial release. Supersedes the four-utility ecosystem model defined in `20260305-002-ecosystem-direction.md`. The vault, catalog, and sync utilities are consolidated into a single metadexer application. shruggie-indexer remains standalone.

Appendix A: Uniform Conventions

The following conventions apply to **all** components in the ShruggieTech ecosystem (both shruggie-indexer and metadexer). They are derived from patterns established and battle-tested during the shruggie-indexer implementation. These conventions are mandatory - not guidelines.

A.1 Documentation Philosophy

All technical specifications and sprint planning documents are written **AI-first, Human-second**. The primary consumers are AI implementation agents operating within isolated context windows during sprint-based development.

- The technical specification for each component is the **single source of truth** for that component's behavioral contract.
- When the specification and the implementation disagree, the specification is presumed correct unless a deliberate amendment has been made.
- Specifications are maintained as living documents alongside the codebase.

A.2 Requirement Level Keywords

All specifications use the keywords defined in RFC 2119:

Keyword	Meaning
MUST / **MUST NOT**	Absolute requirement or prohibition.
SHALL / **SHALL NOT**	Synonymous with MUST / MUST NOT.
SHOULD / **SHOULD NOT**	Strong recommendation. Deviation must be deliberate.
MAY	Truly optional.

These keywords are capitalized when used in their RFC 2119 sense.

A.3 Typographic Conventions

- `Monospace` denotes code identifiers, file paths, CLI flags, configuration keys, and literal values.
- **Bold** denotes emphasis or key terms being defined.
- *Italic* denotes document titles, variable placeholders, or first use of a defined term.
- `§N.N` denotes a cross-reference to a section within the same specification.

A.4 File Encoding and Line Endings

All source files, configuration files, output files, documentation files, and specification documents MUST use:

- **UTF-8 encoding without BOM.**
- **LF (Unix) line endings.**

This is a hard invariant. Output functions MUST use `encoding="utf-8"` explicitly - never rely on platform defaults.

A.5 Python Version and Language Standards

All components target **Python 3.12+** as the minimum version. This enables use of `tomllib` (stdlib TOML parser, 3.11+), modern type hint syntax, and other language features.

A.6 Configuration Architecture

All components use **TOML** as the configuration file format, parsed by Python's `tomllib` module.

Layered override behavior (lowest to highest priority):

1. Compiled defaults (always present; a Python module, not a TOML file).
2. User config directory (platform-specific; see A.7).
3. Project-local config (searched upward from target directory).
4. CLI/API arguments (highest priority).

Configuration objects SHOULD be frozen (immutable) dataclasses. Unknown keys in user-provided TOML MUST be silently ignored (forward compatibility). Invalid values MUST produce clear error messages naming the offending key and value.

A.7 Application Data Directory

Each component resolves its application data directory through a single canonical module. No other module may resolve application data paths independently.

Platform	Base path	Environment variable
Windows	`%LOCALAPPDATA%\shruggie-tech\<component>\`	`LOCALAPPDATA`
Linux	`~/.config/shruggie-tech/<component>/`	`XDG_CONFIG_HOME` (fallback: `~/.config`)
macOS	`~/Library/Application Support/shruggie-tech/<component>/`	*(hardcoded)*

Important: On Windows, all data goes under `%LOCALAPPDATA%` (Local), not `%APPDATA%` (Roaming). Roaming sync is deprecated on Windows 11, and `platformdirs` defaults to Local.

A.8 CLI Interface Conventions

All CLI interfaces use `click` as the argument parser. CLI contracts are the canonical interface - API and GUI layers are secondary.

- Content filtering flags are independent of output destination flags.
- `stdout` stays clean for structured output (JSON). All diagnostics go to `stderr`.
- Destructive operations require explicit opt-in flags.
- `--dry-run` mode is available for any operation with side effects.

A.9 Error Handling and Logging

All components use Python's standard `logging` module:

- Logger names follow the package structure (e.g., `shruggie_indexer.core.entry`, `metadexer.vault.store`).
- Log output goes to `stderr` (console) and optionally to persistent log files under `<app_data_dir>/logs/`.
- Log files use the naming pattern `YYYY-MM-DD_HHMMSS.log`.

Dependency verification pattern:

Category	Failure mode
Required CLI dependency (e.g., `click`)	Hard error with install instructions.
Required external binary (e.g., `exiftool`)	Warning + graceful degradation.
Optional performance dependency (e.g., `orjson`)	Silent fallback to stdlib equivalent.
Development/test dependency (e.g., `pytest`)	Import error at test time only.

A.10 JSON Conventions

- All JSON output uses UTF-8 encoding without BOM.
- Serializers SHOULD use compact formatting for production output.
- `schema_version` is placed first in serialized output by convention.
- `null` is used (not omitted) for explicitly absent values on required fields. Optional fields that are `None` are omitted from output.
- Non-ASCII characters are preserved as literal UTF-8 - not escaped to `\uXXXX` sequences.
- `orjson` is preferred for performance where available, with `json.dumps()` as a silent fallback.

A.11 Testing Strategy

Test suites are organized by test type, not by source module:

Category	Directory	Scope
Unit	`tests/unit/`	Individual functions in isolation.
Integration	`tests/integration/`	Full pipeline, end-to-end.
Conformance	`tests/conformance/`	Output structure against canonical schemas.
Platform	`tests/platform/`	OS-specific behavior.

All tests run with a bare `pytest` invocation. `pyproject.toml` registers custom markers with `--strict-markers`.

A.12 Version Numbering

All components follow semantic versioning (`MAJOR.MINOR.PATCH`). Pre-release versions use suffixes like `-rc1`. The version string lives in a single `_version.py` file. All other references derive from it.

A.13 Packaging and Distribution

Components are **not published to PyPI**. End users download pre-built executables from GitHub Releases.

Packaging stack:

- `pyproject.toml` as the single metadata and dependency declaration file.
- PyInstaller for standalone executables (CLI and GUI as separate build targets).
- GitHub Actions release pipeline triggered on `v*` tag pushes, with matrix builds for Windows (x64), Linux (x64), and macOS (arm64).

Release pipeline stages: Checkout → Test → Build (PyInstaller) → Rename artifacts (version + platform tags) → Upload → Create GitHub Release.

A.14 Documentation Site

All components use **MkDocs** with the **Material for MkDocs** theme. Dark mode (`slate` scheme) is enabled. Required extensions: `admonition`, `pymdownx.details`, `pymdownx.superfences`.

Documentation source lives in `docs/` within each repository. `CHANGELOG.md` is auto-copied to the docs site during CI build. Deployed via GitHub Pages.

Appendix B: Specification Status Lifecycle

Status	Meaning
`DRAFT`	Under active development. Sections may be incomplete.
`REVIEW`	Believed complete; undergoing review.
`APPROVED`	Reviewed and accepted as the implementation target.
`AMENDED`	Modified after initial approval to reflect post-release changes.

Appendix C: Repository Organization

shruggie-indexer (existing)

```
shruggie-indexer/
├── .archive/
├── .github/workflows/
├── docs/
├── scripts/
├── src/shruggie_indexer/
├── tests/
├── .gitignore
├── CHANGELOG.md
├── LICENSE
├── mkdocs.yml
├── pyproject.toml
├── README.md
└── shruggie-indexer-spec.md
```

metadexer (planned)

```
metadexer/
├── .archive/
├── .github/workflows/
├── docs/
├── scripts/
├── src/metadexer/
│   ├── __init__.py
│   ├── _version.py
│   ├── cli.py
│   ├── vault/
│   │   ├── __init__.py
│   │   ├── store.py          # core put/get/head/verify logic
│   │   ├── backends/
│   │   │   ├── local.py      # local filesystem backend
│   │   │   └── s3.py         # S3-compatible backend
│   │   └── ...
│   ├── catalog/
│   │   ├── __init__.py
│   │   ├── ingest.py         # IndexEntry ingestion
│   │   ├── search.py         # query interface
│   │   ├── backends/
│   │   │   ├── sqlite.py
│   │   │   └── postgres.py
│   │   └── ...
│   └── sync/
│       ├── __init__.py
│       ├── pipeline.py       # orchestration logic
│       ├── plan.py           # Sync Plan generation
```

```
│       └── ...
├── tests/
├── .gitignore
├── CHANGELOG.md
├── LICENSE
├── mkdocs.yml
├── pyproject.toml
├── README.md
└── metadexer-spec.md
```

Appendix D: Archive Directory and Sprint Document Conventions

Archive Directory Naming

All files in `.archive/` follow a strict naming pattern:

```
<YYYYmmdd>-<ZZZ>-<title>.<ext>
```

Component	Description
`YYYYmmdd`	Date the document was created.
`ZZZ`	Three-digit zero-padded increment. Resets to `001` on each new date.
`title`	Lowercase-hyphenated descriptive title.
`ext`	File extension.

Sprint Document Format

Sprint planning documents follow a five-section structure:

1. **Header block** - project name, repository, author, date, target release, audience, predecessor reference.
2. **Purpose and ecosystem context** - what this sprint accomplishes and why.
3. **Implementation ordering** - strict dependency-ordered sequencing with rationale.
4. **Work item sections** - each section is a self-contained sprint suitable for a single AI coding agent context window, containing: problem statement with evidence, root cause analysis (for bug fixes), required changes with explicit file paths, affected file matrix, spec cross-references, acceptance criteria, and mandatory verification commands.
5. **Specification update directive** - always the last section; reflects all changes into the authoritative spec.

Each sprint document is paired with a `_TEMPLATE.txt` prompt file for AI coding agent sessions.

AI Agent Session Discipline

- Agents MUST NOT trust prior implementation work without independent verification.
- Agents MUST use grep-based evidence collection to verify codebase state before making changes.
- Agents MUST verify acceptance criteria against actual runtime behavior, not assumed correctness from code inspection alone.
- Each sprint section is self-contained.

Appendix E: Implementation Guidance

When generating code for any component, the following constraints are invariant:

- Maintain strict separation of module responsibilities as defined above. A module must not absorb logic belonging to a neighbor.
- Avoid shared mutable state across modules.
- Favor explicit JSON contracts over implicit in-process coupling.
- Design CLI interfaces first; API and daemon layers are secondary.
- Keep API layers as thin transports over CLI logic. Do not implement independent behavior at the API layer.
- All destructive operations require explicit opt-in flags.
- All files produced by tooling must use UTF-8 encoding without BOM.
- Evidence-based debugging: runtime logs and filesystem state are the primary diagnostic tools.
- No silent data loss: all destructive operations require explicit opt-in. Duplicate removal must preserve complete provenance.

Appendix F: Composition Rules

No module may:

- Recompute identity unless performing an explicit integrity verification.
- Rewrite or amend an IndexEntry outside of the indexer's own operation.
- Implicitly delete content from any backend.
- Implicitly migrate content between backends.

Cross-module interaction within metadexer occurs through defined internal APIs. The indexer communicates with metadexer exclusively through the IndexEntry JSON contract.